14-06-2026

Agenda:

ML-7

- Regularization
- practicals
- Bias & variance
- logistic Regression

Regularization:

problem:

Age $\rightarrow x_1$ ↓ numerical

City $\rightarrow x_2 \rightarrow$ OHE ↓ categorical

- Mumbai
- Delhi

cibil-score $\rightarrow y$

Age Mumbai Delhi $\rightarrow (m_{age} \cdot x_1) + (m_{mub} \cdot x_2) + (m_{del} \cdot x_3) + c \rightarrow \hat{y}$

x_1	x_{2m}	x_{3d}	y
23	1	0	700
42	1	0	700
54	0	1	600
34	0	1	600

SGD Regressor:

case:1

$m_{age} = 0$

$m_{mum} = 100$

$m_{del} = 0$

$c = 600$

→ person A ← delhi

Age = 34, mum = 0, del = 1

$(34\times0) + (100\times0) + (1\times0) + 600$

$= 600$

→ person B ← mumbai

Age = 34, mum = 1, del = 0

$(34\times0) + (1\times100) + (0\times0) + 600$

$= 700$ (perfect score)

case2:

$m_{age} = 0$

$m_{mum} = 1700$

$m_{del} = 1600$

$c = -1000$

person A ← del

(34, 0, 1)

$(34\times0) + (0\times1700) + (1\times1600) + -1000$

$= 0 + 0 + 1600 - 1000$

$= 600$ (perfect)

person B ← mum

$0 + 1700 + 0 + -1000$

$= 700$ (perfect)

issue:

$(0, 10, 1)$ $(0, -999999, 100000)$

→ model can wonder into universe of parameters, because its only metric for success for is error.

↓ direction for solution (Regularization)

→ keep parameter explosion under control

↳ speeds up the convergence (final value)

↓ apply the solution

tiny bit of error which makes the model slightly less accurate, that prevents overfitting

↓ generalised model

all solution → large β → X
all solution → medium β → X
(large and medium: eliminating possible solution)
all solution → small β → optimal

Regularization:

λ = control the penalty

L1 (Lasso):

$$\text{loss}_{new} = \text{Modified MSE} + \lambda \sum |\beta_i|$$

$$= \frac{1}{2n} \sum (y - \hat{y})^2 + \lambda \sum |\beta_i|$$

derivative

$$= \underbrace{\frac{1}{n} X^T (\hat{y} - y)}_{\text{Error term}} + \underbrace{\lambda \cdot \text{sign}(\beta_i)}_{\text{L1 penalty term}}$$

if $\beta > 0 \rightarrow +1$
$\beta < 0 \rightarrow -1$
$\beta == 0 \rightarrow$ undefined

L2 (Ridge)

$$\text{Loss}_{new} = \text{Modified MSE} + \lambda \sum \beta_i^2$$

$$\text{derivative} = \underbrace{\frac{1}{n} X^T(\hat{y} - y)}_{\text{error term}} + \underbrace{2 \cdot \lambda \cdot \beta}_{\text{L2 penalty term}}$$

L1	L2
	→ The higher the weight, the strong the change
start	**start**
→ $w = 10$	→ $w = 10$
→ $\text{sign}(10) \to +ve \to 1$	→ $2 \cdot 10 = 20$
End	**End**
→ $w = 0.0001$ $\lambda = 0.01$	$w = 0.0001$
→ $\text{sign}(0.0001) \to +ve \to 1$	→ 2×0.0001 → 0.0002 large
→ 0.01×1 → 0.01	→ $2 \times 0.01 \times 0.0001$ large
→ Feature weight can become 0 ↓ Automatic feature Elimination	↓ Feature weight never converges to 0

L1 → messy data, 200 features, 15 to 20 features

L1 → 15 to 20, other will be converged to 0

L2 → safest option

→ believe all the feature might have some importance

→ overfitting

Year of education	Age

share the workload

Interview:

(1) which regularization to use for a simple or small model & tell my why?

→ L1 regularization

(2) Many columns useful

→ Ridge Regularization

(3) keep Highly coorelated columns:

→ L2

(4) High correlated columns: L1

Technique : 3

$\longrightarrow$ Elastic Net (L1_ratio = 0.3)

L1 → 30% of penalty force from L1

L2 → 70% comes L2

$$\text{loss}_{new} = \text{Modified MSE} + \underbrace{\lambda \sum \beta_i^2}_{70\%} + \underbrace{\lambda \sum |\beta_i|}_{30\%}$$

```
SGDRegressor (
    penalty = 'L1', 'L2', 'elasticnet',
    l1_ratio = 0.5
    alpha = 0.0001
)
```

OHE :

Blue
Green
Red

LE →

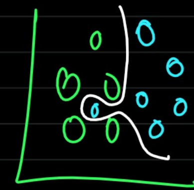
B	G	R
1	0	0
0	1	0
0	0	1

multicolin

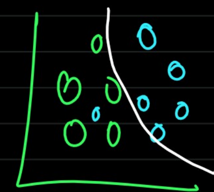
1 → all

0 → one non-zero element

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1



→ without Regularization



→ with Regularization

Blue
Green → OHE → training →
Red

obj.transform() → error

purple → test

Handle

handle_unknown = 'ignore' , drop= "first"

B → OHE → 1 0 0
G → OHE → 0 1 0
R → OHE → 0 0 1

L2
Elasticnet

P → OHE → 0 0 0
V → OHE → 0 0 0 → OHE
P → OHE → 0 0 0

training → .fit

learn & return/apply → fit_transform

return from learned parameter / apply (preprocessors) → transform

sklearn
ML libraries

return/apply from the trained model → predict